Reinforcement Learning for Mathematical Reasoning in Small Language Models: A Comparison of REINFORCE, PPO, and GRPO

Ahmad Shaker **Abdullah Elganainy** **Karim Abousamra**
CS 4453

Abstract

We investigate whether reinforcement learning (RL) can improve multi step mathematical reasoning in a small, resource constrained language model. Using Qwen2.5-1.5B-Instruct with LoRA adapters trained on the GSM8K benchmark, we implement and compare three policy gradient algorithms: REINFORCE with an exponential moving average (EMA) baseline, Proximal Policy Optimization (PPO), and Group Relative Policy Optimization (GRPO). Our reward function is entirely rule based: rewarding with a +1.0 for correct answers, +0.1 for well structured but incorrect responses, and smaller partial credits otherwise, eliminating the need for a learned reward model. The base model already achieves 0.505 accuracy before any RL training, establishing a strong starting point. All three algorithms improve over this baseline: REINFORCE converges fastest, reaching 0.590 accuracy at step 100 before declining to 0.530 by step 200; PPO is the most volatile in practice, as our simplified EMA baseline undermines the stability its clipped surrogate is designed to provide, yet it still recovers to 0.540; and GRPO achieves the best final accuracy at 0.615 while requiring no critic network. These results demonstrate that rule based RL loop can meaningfully improve math reasoning in a sub 2B parameter model on common hardware.

1. Introduction

1.1 What Is the Problem?

Large language models (LLMs) have achieved impressive results across many natural language tasks, yet they continue to struggle with **multi step mathematical reasoning**. Even models with hundreds of billions of parameters routinely fail on grade school math that any middle school student could handle, making mathematical errors mid chain that cascade into completely wrong final answers. The failure mode is not knowledge, the models possess the relevant facts, but the ability to execute a reliable, step by step computation plan.

This problem is practically significant: mathematical reasoning is a proxy for structured, verifiable logical thinking, and improvements here generalise to domains such as code generation, scientific calculation, and financial analysis. GSM8K (Cobbe et al., 2021), the benchmark we use, consists of 8,500 grade school word problems requiring 2–8 arithmetic steps. Despite the apparent simplicity of this distribution, achieving high accuracy has historically required very large models.

1.2 Why Do Existing Techniques Fall Short?

Supervised fine tuning (SFT) on chain of thought (CoT) demonstrations (Wei et al., 2022) improves reasoning but is limited by the quality and diversity of available annotated solutions. The model learns to "imitate" training solutions rather than to "reason" towards correct answers. Small models fine tuned using SFT quickly hit a performance ceiling because they memorise surface patterns instead of internalising the underlying logic.

Reinforcement learning from human feedback (RLHF, Ouyang et al., 2022) can push past this ceiling by directly optimising for outcome correctness, but its standard implementation requires a **learned reward model** trained on human preference data, an expensive and noisy component. In the mathematical domain, correctness is binary and automatically verifiable, making a learned reward model unnecessary and potentially harmful (it may assign high reward to correct sounding but wrong answers).

Finally, most RL-for-reasoning work (e.g., Lightman et al., 2023; DeepSeek-R1, 2025) operates at the scale of 7B–70B parameter models. Whether RL can meaningfully improve a sub-2B model under memory and compute constraints typical of a laptop or free tier cloud instance remains an open question, and is what this project addresses.

1.3 Our Approach

We propose to train *Qwen2.5-1.5B-Instruct* using three policy gradient RL algorithms: REINFORCE, PPO, and GRPO, with a **purely rule based reward function** that awards full credit for correct answers, partial credit for well structured incorrect answers, and zero for incoherent outputs. LoRA adapters keep the trainable parameter count below 2% of total weights, enabling training on a single 12 GB GPU or even a CPU. The key intuition is: *if reward is verifiable and automatic, we can skip the reward model entirely and let outcome feedback directly shape the policy.*

2. Techniques to Tackle the Problem

2.1 Review of Prior Work

Chain-of-Thought Prompting (Wei et al., 2022). Wei et al. showed that prompting large models to produce step by step reasoning ('chain of thought') before giving a final answer dramatically improves accuracy on multi step tasks, including arithmetic. This was the first demonstration that intermediate reasoning steps are a key enabler of mathematical correctness in LLMs. Our prompt template directly adopts this insight.

GSM8K Benchmark (Cobbe et al., 2021). Cobbe et al. released GSM8K alongside a study of training verifiers, small classifiers that score model outputs and are used to select the best from multiple samples. Their work established that outcome based verification (rather than teacher forced imitation) is a productive training signal for math. Our rule based reward function is a deterministic instantiation of this idea.

Let's Verify Step by Step (Lightman et al., 2023). Lightman et al. demonstrated that process reward models (PRMs), which score every intermediate reasoning step rather than only the final answer, outperform outcome reward models on difficult math benchmarks. While we use an outcome reward for simplicity and reproducibility, their finding motivates the partial credit

terms in our reward function (rewarding the presence of real reasoning steps, not just correct final answers).

PPO (Schulman et al., 2017). PPO replaced the hard KL constraint of TRPO with a clipped surrogate objective, making trust region policy optimisation practical. It became the standard RL algorithm in RLHF pipelines (Ouyang et al., 2022) due to its empirical stability. Our PPO implementation follows the original clipped surrogate formulation.

DeepSeekMath / GRPO (Shao et al., 2024). Shao et al. introduced GRPO specifically for mathematical reasoning. By sampling a *group* of K outputs per prompt and normalising rewards within the group, GRPO estimates per prompt advantages without a value network. The authors showed that this critic free approach matches PPO accuracy on mathematical tasks while using roughly half the GPU memory, a critical advantage at small scale.

DeepSeek-R1 (DeepSeek-AI, 2025). DeepSeek-R1 demonstrated that RL training with rule based verifiable rewards (without any SFT warm start) can produce sophisticated chain of thought reasoning and even emergent self correction in very large models. While we operate at a fraction of their scale, their result validates the premise that verifiable outcome RL is a principled and powerful approach to reasoning improvement.

REINFORCE with Baselines (Williams, 1992). Williams' foundational policy gradient theorem underpins all three algorithms we study. In particular, the use of a baseline (any function independent of the action) to subtract from the return reduces gradient variance without introducing bias. Our REINFORCE implementation uses an exponential moving average (EMA) of past rewards as the baseline.

2.2 Our Technique: Stepped Rule-Based Reward with LoRA

Our primary methodological contribution is a **stepped reward function** designed to provide dense training signal even in early training when the model rarely produces correct answers:

$$R = 1.0 \quad \textit{if answer correct}$$
$$R = 0.1 \quad \textit{if answer wrong, but has '####' AND genuine reasoning}$$
$$R = 0.02 \quad \textit{if answer wrong, but has '####' OR reasoning (not both)}$$
$$R = 0.0 \quad \textit{otherwise}$$

'Genuine reasoning' is detected by the presence of arithmetic operators (+, −, *, /), step markers (numbered or bulleted lines), or reasoning connectives ('therefore', 'thus', 'hence', etc.). This three tier detection is harder to game than a simple length threshold.

The partial rewards (0.1 and 0.02) are intentionally small relative to the correct answer reward (1.0), ensuring that the model cannot substantially improve its total expected reward by gaming format alone. The partial credit serves to bootstrap early training: a model that has never seen a correct reward still receives gradient signal for producing well structured reasoning.

All algorithms use LoRA (Hu et al., 2021) with rank 16 applied to the query, key, value, and output projection matrices. This limits trainable parameters to roughly 0.28% of the total model weights, enabling training within 12 GB VRAM.

2.3 Algorithm Descriptions

REINFORCE with EMA Baseline. At each step, we sample a mini batch of prompts, generate one response per prompt, compute rewards, subtract the EMA baseline, and update the policy via:

$$L_REINFORCE = -E[(R - b) \cdot \log \pi(a \mid s)]$$

where b is the EMA baseline updated as $b \leftarrow 0.85 \cdot b + 0.15 \cdot \text{mean}(R)$. This is the simplest possible policy gradient estimator, high variance but computationally cheap, requiring only one forward and one backward pass per prompt per step.

PPO with Clipped Surrogate Objective. PPO generates responses, records old log probabilities, and performs multiple mini epochs of updates using the clipped objective:

$$L_PPO = -E[\min(r(\theta) \cdot A, \text{clip}(r(\theta), 1-\varepsilon, 1+\varepsilon) \cdot A)]$$

where $r(\theta) = \pi_new(a|s) / \pi_old(a|s)$ and $\varepsilon = 0.2$. We use a simplified EMA value baseline rather than a full critic network, keeping memory requirements comparable to REINFORCE. Two PPO mini epochs are performed per batch, with a cosine learning rate schedule and 50-step linear warmup.

GRPO with Group-Relative Advantages. GRPO samples a group of K=4 responses per prompt, computes the within-group advantage:

$$A_i = (R_i - \text{mean}(R_group)) / (\text{std}(R_group) + \varepsilon)$$

and adds a KL penalty against initial LoRA weights to keep the policy from diverging:

$$L_GRPO = -E[A \cdot \log \pi(a|s)] + \lambda \cdot KL(\pi \,||\, \pi_ref)$$

When all group rewards are zero (common early in training), we assign a uniform negative advantage of −1.0 to encourage exploration rather than skipping the update entirely, a bug fix over naive implementations. Gradient updates are accumulated across groups before a single optimiser step to reduce peak VRAM.

3. Evaluation

3.1 Experimental Setup

Table 1: Experimental configuration.

Component	Details
Base model	Qwen2.5-1.5B-Instruct (bfloat16)
LoRA config	Rank 16, α=32, dropout=0.05, targets: q/k/v/o_proj
Training steps	200 per algorithm
Batch size	4 prompts (GRPO: 4 prompts × 4 responses)
Learning rate	5×10^{-5} with 50-step warmup + cosine decay
Evaluation	200 random test problems every 100 steps
Hardware	NVIDIA RTX 4070 GPU, 12.9 GB VRAM (or CPU fallback)
Dataset	GSM8K: 7,473 train / 1,319 test

Each algorithm is initialised from the same base model with freshly reset LoRA weights to ensure a fair comparison. The untrained base model establishes a baseline accuracy against which all three are measured. With a baseline accuracy of 0.505.

3.2 Results

Table 2: Empirical results from the 200-step training run with Qwen2.5-1.5B-Instruct. Accuracy values read directly from the Test Accuracy plot.

Algorithm	Acc. @ Step 100	Acc. @ Step 200	Avg Reward (final)	Trend
REINFORCE	0.590	0.530	~0.786	Peaks early, slight decline
PPO	0.470	0.540	~0.571	Volatile, recovers late
GRPO	0.575	0.615	~0.764	Best final accuracy

Figure 1: Reward curves (left) and test accuracy (right) for all three algorithms over 200 training steps.

Training Stability. Contrary to our initial hypothesis, PPO did *not* exhibit the smoothest reward curve. As shown in Figure 1 (left), PPO's mean reward was highly volatile throughout training, dropping as low as ~0.40 around step 125 before partially recovering. REINFORCE and GRPO both trended more steadily upward, reaching smoothed rewards in the 0.75–0.90 range by step 200. The likely explanation is that our simplified EMA value baseline in PPO, used in place of a full critic network to save memory, provides poor advantage estimates, undermining the stability benefit that the clipped surrogate objective is designed to deliver. A full actor critic PPO with a learned value head would likely behave more as expected.

Final Accuracy. At step 200, GRPO achieved the highest test accuracy at 0.615, edging out PPO (0.540) and REINFORCE (0.530). Interestingly, REINFORCE started strongest, achieving 0.590 at step 100, but declined over the second half of training, suggesting that the high variance gradient updates eventually caused the policy to drift away from good solutions. PPO improved monotonically but started from the lowest baseline (0.470 at step 100), reflecting the instability visible in its reward curve. GRPO's final accuracy being highest is consistent with its group relative advantage estimates providing the most informative gradient signal per update.

Memory and Compute Cost. REINFORCE and GRPO had comparable peak VRAM usage (~4–6 GB in bfloat16). PPO required slightly more due to storing old log-probabilities and performing two mini epochs per batch. GRPO's per step wall clock time was higher than REINFORCE because it generates K=4 responses per prompt, but its total compute per accuracy point was competitive given its superior final accuracy. On CPU, all three algorithms ran to completion, with REINFORCE being fastest due to single response generation.

Table 3: Algorithm comparison.

Property	REINFORCE	PPO	GRPO	Winner
Acc. @ step 200	0.530	0.540	0.615	GRPO
Reward stability	Medium	Low (volatile)	Medium-High	GRPO
Critic network	None	EMA only	None	Tie (R/G)
Responses/prompt	1	1	K=4	REINFORCE/PPO
Peak VRAM	Low	Medium	Low-Med	REINFORCE
Implementation	Simple	Complex	Medium	REINFORCE

3.3 Qualitative Analysis

Contrary to expectations, the base model was already capable of structured reasoning before any RL training, the notebook's sample output shows the model correctly solving a multi step problem with explicit arithmetic and a properly formatted #### delimiter, achieving 0.505 accuracy on 200 test problems. This means RL is not teaching the model to reason from scratch; it is selectively reinforcing reasoning patterns the model already possesses but applies inconsistently. After 200 steps, accuracy shifts ranged from 0.025 (REINFORCE declining) to +0.090 (GRPO improving), a modest but meaningful signal given the already-high baseline.

The diverging trajectories across algorithms are the most informative qualitative finding. REINFORCE's early peak and subsequent decline suggests the high variance EMA baseline allows early lucky updates to overfit the policy, which then degrades under accumulated noise. PPO's volatile reward curve, dropping as low as ~0.40 around step 125 before partially recovering, reflects the lack of a proper critic: without accurate value estimates, the clipped surrogate cannot enforce a meaningful trust region. GRPO's steady upward trend, by contrast, reflects a more stable training dynamic: normalising rewards within a group of K=4 responses per prompt provides a per step advantage signal that is both lower variance than the EMA baseline and more informative than a single sample estimate, without requiring any additional network parameters.

4. Conclusion

4.1 Can RL Effectively Tackle This Problem?

Yes, all three RL algorithms improved over the untrained baseline, and GRPO achieved the highest final test accuracy (0.615) after 200 training steps. The results confirm that **rule based outcome rewards are a viable and practical training signal for math reasoning at small scale**, without requiring a learned reward model.

The most surprising finding was PPO's instability: our simplified EMA baseline in place of a full critic network undermined the trust region benefit that makes PPO attractive in theory. GRPO, by contrast, delivered strong final accuracy through group relative normalisation without requiring any value network. REINFORCE peaked earliest (0.590 at step 100) but declined over training, consistent with cumulative high variance gradient updates gradually degrading the policy.

GRPO is our recommended algorithm for this setting: it matches or exceeds PPO's final accuracy, requires no critic network, and is more memory efficient. REINFORCE remains useful as a fast converging baseline in compute-constrained settings where the first 100 steps matter more than eventual asymptotic performance.

4.2 Limitations

- Results are based on 200 training steps, which is 7.5× fewer than the full 1,500-step experiment described in our proposal. Publication quality conclusions require the full run.
- We use a simplified EMA value baseline in PPO rather than a full learned critic. A proper value network would likely improve PPO's advantage estimates and final accuracy.

- Our evaluation metric (exact numerical match) does not capture partial reasoning correctness. A step level evaluation following Lightman et al. (2023) would provide richer insight.
- We did not test generalisation beyond GSM8K. Whether RL induced improvements on grade school arithmetic transfer to harder benchmarks (MATH, AIME) is an open question.

4.3 Future Research Directions

- **Scale the training run.** Running 1,500 steps with the 1.5B model on a T4 GPU is the immediate next step. We expect the algorithm ranking to be stable but the absolute accuracy gaps to widen, providing clearer conclusions.
- **Process reward models (PRMs).** Following Lightman et al. (2023), training a PRM to score intermediate steps could provide denser reward signal and further accelerate learning, especially for problems requiring 6–8 steps.
- **Self-play / iterative DPO.** Alternating between RL training and preference data collection from the trained model (as in SPIN or iterative DPO) could provide a curriculum that continually challenges the model with its own failure modes.
- **Curriculum learning.** Sorting training problems by difficulty (e.g., by number of steps or intermediate complexity) and introducing harder problems progressively may improve convergence speed and reduce variance.
- **Multi-task transfer.** Pre-training the LoRA adapters on a diverse mathematical corpus before RL could improve the quality of initial explorations and reduce the reward sparsity problem in early training.

References

Cobbe, K., Kosaraju, V., Bavarian, M., et al. (2021). *Training Verifiers to Solve Math Word Problems.* arXiv:2110.14168.

DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv:2501.12948.

Hu, E., Shen, Y., Wallis, P., et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685.

Lightman, H., Kosaraju, V., Burda, Y., et al. (2023). *Let's Verify Step by Step.* arXiv:2305.20050.

Ouyang, L., Wu, J., Jiang, X., et al. (2022). *Training Language Models to Follow Instructions with Human Feedback.* arXiv:2203.02155.

Schulman, J., Wolski, F., Dhariwal, P., et al. (2017). *Proximal Policy Optimization Algorithms.* arXiv:1707.06347.

Shao, Z., Wang, P., Zhu, Q., et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.* arXiv:2402.03300.

Wei, J., Wang, X., Schuurmans, D., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* NeurIPS 2022.

Williams, R.J. (1992). *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning.* Machine Learning, 8(3-4), 229–256.